

第 3 集 · 约 17 分钟

把 Rust 送进浏览器： WASM 与 unsafe

JS 和 WASM 之间只能传数字，1 MiB 的文件块
怎么过去？

配套阅读：书的第 3 章



这一集你会学到

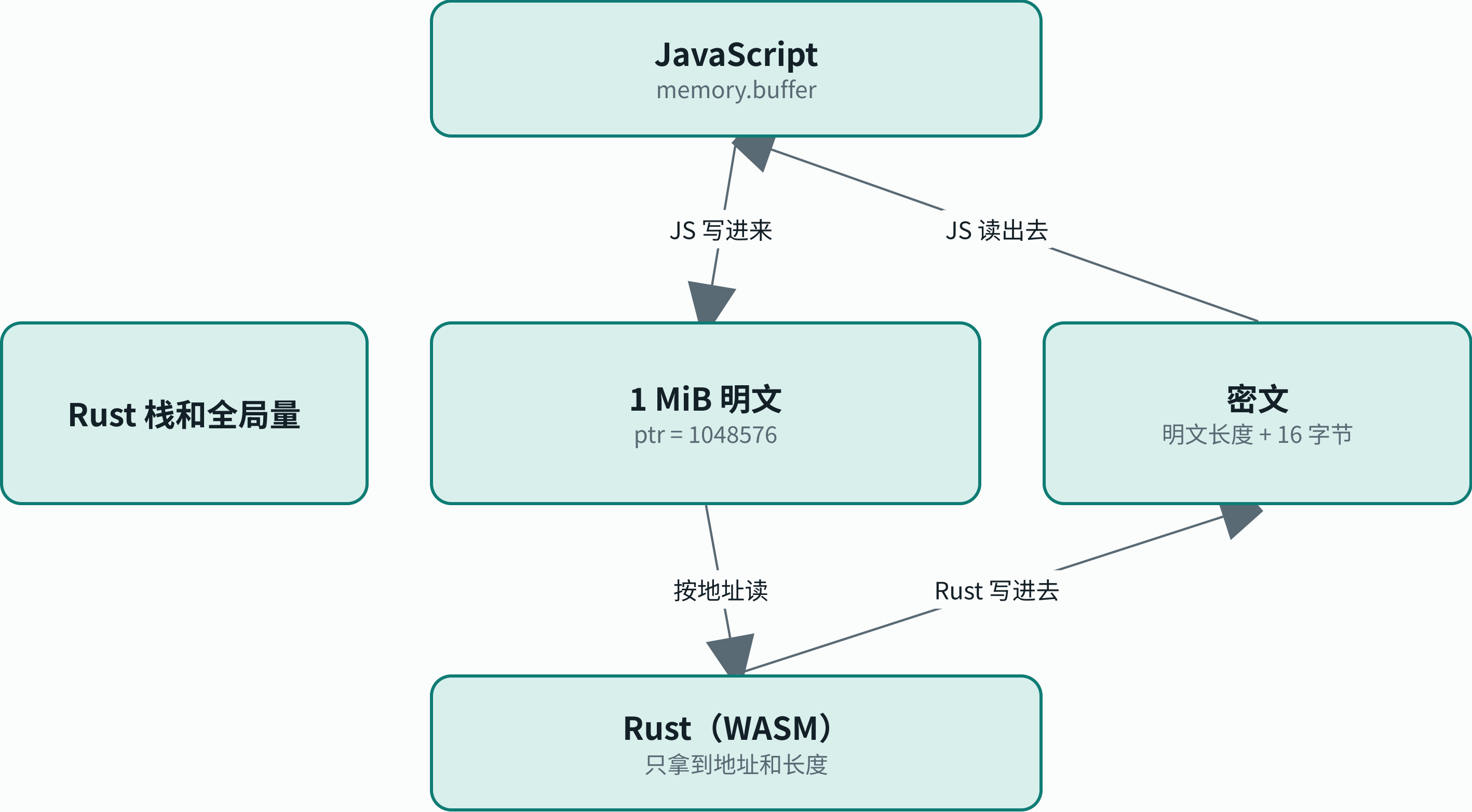
1. 线性内存：JS 和 WASM 共用的一大块字节数组
2. 裸指针和 unsafe：编译器管不了的地方，由人写 SAFETY 担保
3. `std::mem::forget`：把内存交给 JS 保管，用完再收回来
4. `thread_local + Option::take`：把“只能用一次”写进代码结构
5. JS 那边的两个坑：内存扩容后视图失效，64 位整数要拆开传

一个问题

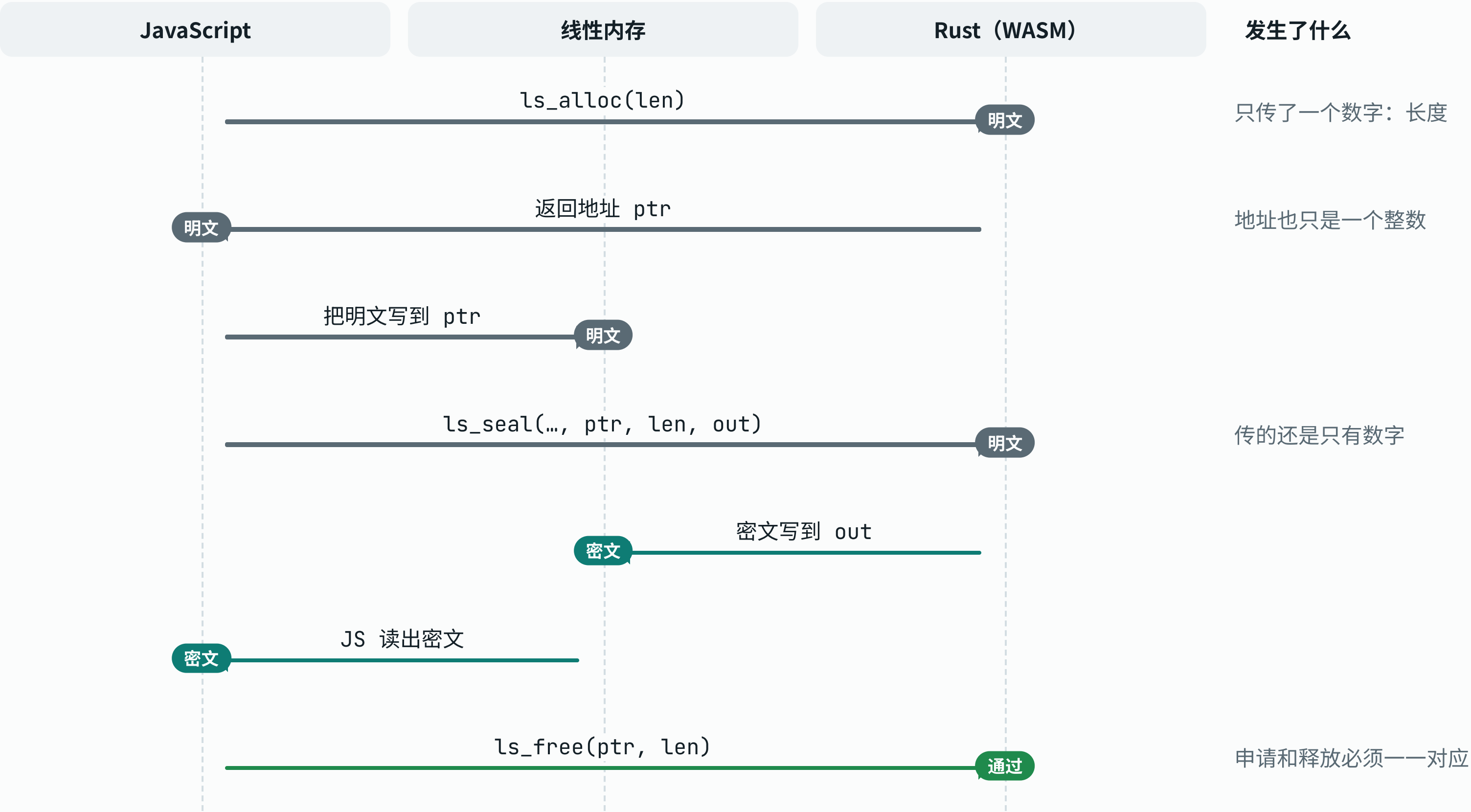
JS 和 WASM 之间只能传数字。那 1 MiB 的文件块，怎么交给 Rust 去加密？

传地址。两边共用一块内存：JS 把数据写进去，再把“地址”和“长度”这两个数字交给 Rust。

线性内存：共享的大数组



传一块字节，分四步



ctypes 也这么干

Python · ctypes 调 C 动态库

```
1 import ctypes
2 lib = ctypes.CDLL("crypto.dll")
3 buf = ctypes.create_string_buffer(1024) # 申请一块 C 内存
4 lib.seal(key, ctr_hi, ctr_lo, buf, len(buf)) # 只能传指针和长度
5 result = buf.raw # 把结果拷回 Python
```

JS 调 WASM

```
1 const ptr = x.ls_alloc(len); // 申请一块 WASM 内存
2 view(ptr, len).set(data); // 把数据写进去
3 x.ls_seal(k, hi, lo, a, aadLen, ptr, len, out); // 只能传指针和长度
4 const result = view(out, len + 16).slice(); // 把结果拷出来
5 x.ls_free(ptr, len);
```

把地址变回切片

crates/lanshare-wasm/src/lib.rs

```
1  /// SAFETY: 调用方保证 `ptr` 指向至少 `len` 字节的有效内存 (`len == 0` 时可以是任意值)。  
2  unsafe fn input<'a>(ptr: *const u8, len: usize) -> &'a [u8] {  
3      if len == 0 {  
4          &[]  
5      } else {  
6          // SAFETY: 由本函数的调用约定保证。  
7          unsafe { std::slice::from_raw_parts(ptr, len) }  
8      }  
9  }
```

申请和释放

crates/lanshare-wasm/src/lib.rs

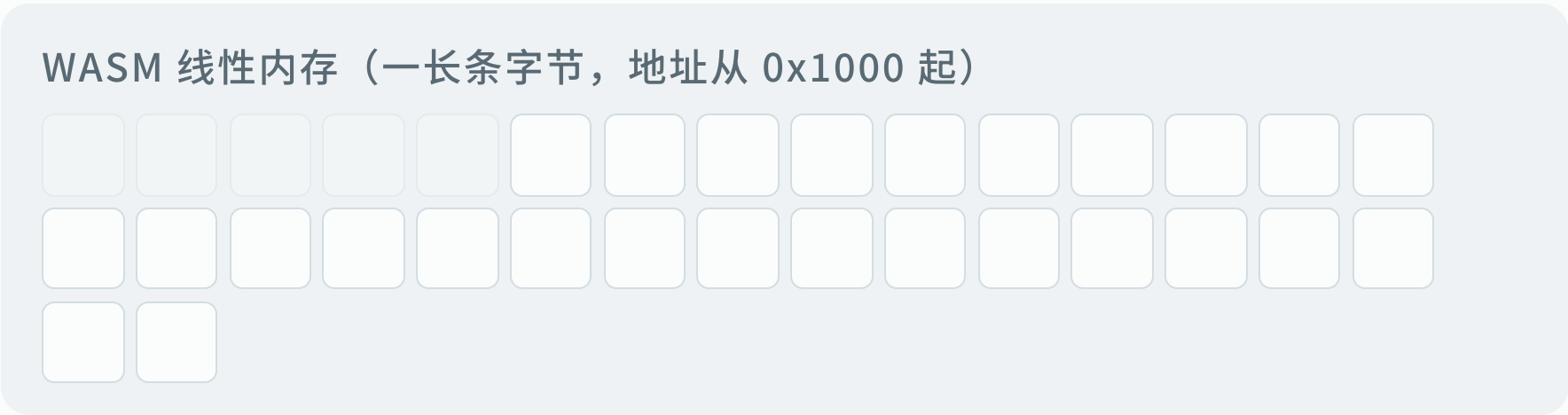
```
1  /// 申请 `len` 字节，返回指针给 JS 写数据。用完必须用同样的 `len` 调 [`ls_free`].
2  #[unsafe(no_mangle)]
3  pub extern "C" fn ls_alloc(len: usize) -> *mut u8 {
4      let mut buf = Vec::<u8>::with_capacity(len.max(1));
5      let ptr = buf.as_mut_ptr();
6      std::mem::forget(buf);
7      ptr
8  }
9
10 /// 释放 [`ls_alloc`] 申请的内存。
11 ///
12 /// # Safety
13 /// `ptr` 和 `len` 必须正好来自一次 `ls_alloc(len)`，且只释放一次。
14 #[unsafe(no_mangle)]
15 pub unsafe extern "C" fn ls_free(ptr: *mut u8, len: usize) {
16     // SAFETY: ptr 来自 Vec::with_capacity(len.max(1))，长度按 0 重建即可，析构时只释放容量。
17     drop(unsafe { Vec::from_raw_parts(ptr, 0, len.max(1)) });
18 }
```

动手：JS 调 WASM

```
1 const ptr = x.ls_alloc(len);           // 1. 在 WASM 内存里申请一块
2 view(ptr, len).set(data);              // 2. 把数据拷进去
3 x.ls_seal(k, hi, lo, a, aadLen, ptr, len, out); // 3. 只传指针和数字
4 const result = view(out, len + 16).slice(); // 4. 把结果拷出来
5 x.ls_free(ptr, len);                   // 5. 用完释放
```

JavaScript 这边

```
data = [68 65 6c 6c 6f] ("hello")
result = 21 字节密文
```



ls_free 释放。忘了这一步，WASM 的内存就会越用越多。和 Python 用 ctypes 调 C 一模一样。

状态只能用一次

crates/lanshare-wasm/src/lib.rs

```
1  thread_local! {
2      /// 进行中的 PAKE 状态。WASM 是单线程的，thread_local 就是全局表。
3      static PAKES: RefCell<Vec<Option<proto::PakeState>>> = const { RefCell::new(Vec::new()) };
4  }
5  // ...
6  pub unsafe extern "C" fn ls_pake_finish(handle: u32, peer: *const u8, peer_len: usize, key_out: *mut u8) -> i32 {
7      let state = PAKES.with_borrow_mut(|table| {
8          let index = (handle as usize).checked_sub(1)?;
9          table.get_mut(index)?.take()
10     });
11     let Some(state) = state else { return 1 };
12     // ...
13 }
```

踩坑：JS 这边

Rust 写对了还不够，JS 这边也有两个坑

坑 1：线性内存扩容后，之前创建的 Uint8Array 读出来全是空的
扩容后原来的 ArrayBuffer 会被“分离”（detached），旧视图长度变成 0

修法：不缓存视图，每次现取
`const view = (ptr, len) => new Uint8Array(memory.buffer, ptr, len);`

坑 2：计数器是 u64，老版本的手机浏览器不支持在 JS 和 WASM 之间直接传 64 位整数
修法：拆成高低两个 32 位传，Rust 那边再拼回来
`(u64::from(hi) << 32) | u64::from(lo)`

小而快，三方一致

没用 wasm-bindgen，胶水代码自己写
不用额外装版本必须严格对上的 wasm-bindgen-cli；只导出十几个函数，手写一共一百多行

lanshare.wasm 只有 95.7 KB
浏览器里跑 Python 要用 Pyodide，光解释器就有大约 10 MB

三份独立实现，结果完全一致
OpenSSL 生成期望值，原生 RustCrypto、WASM 里的 RustCrypto 都对得上；crypto.test.mjs 8 项全过

WASM 加密 382 MiB/s，解密 388 MiB/s
i9-12900H，Node 24 实测，远高于 WiFi 的实际速度

选择题

为什么不能把 JS 的 `Uint8Array` 直接传给 WASM 函数？

A 可以传，只是要慢一些

B `Uint8Array` 太大，超过了 WASM 参数的长度上限

C WASM 函数的参数只能是数字，字节要先写进线性内存，再传地址和长度

D 先转成字符串就能传了

选择题

为什么不能把 JS 的 Uint8Array 直接传给 WASM 函数？

A 可以传，只是要慢一些

传不了。WASM 函数的参数只认数字，跟快慢没关系。

B Uint8Array 太大，超过了 WASM 参数的长度上限

不是大小的问题，哪怕只有 1 个字节也传不了，参数只能是整数和浮点数。

C WASM 函数的参数只能是数字，字节要先写进线性内存，再传地址和长度

对。这就是 ls_alloc、写入、调用、读出、ls_free 这一整套的由来。

D 先转成字符串就能传了

字符串也传不了，WASM 函数只认数字。字符串同样要编码成字节，写进线性内存。

选择题

把 `ls_alloc` 里的 `std::mem::forget(buf);` 删掉，会怎样？

A 会内存泄漏

B `buf` 在函数返回时就被释放，JS 拿到的指针指向已经释放的内存

C 没影响，Rust 会等 JS 用完再释放

D 编译不过，编译器会提醒你漏了 `forget`

选择题

把 `ls_alloc` 里的 `std::mem::forget(buf);` 删掉，会怎样？

A 会内存泄漏

正好相反。`forget` 才是“不释放”，删掉它，内存是释放得太早。

B `buf` 在函数返回时就被释放，JS 拿到的指针指向已经释放的内存

对，这叫悬垂指针。JS 往里写数据会破坏 Rust 的堆，属于未定义行为。`forget` 把所有权交给 JS，直到 `ls_free` 再收回。

C 没影响，Rust 会等 JS 用完再释放

Rust 不知道 JS 什么时候用完。没人告诉它，它就按作用域的规矩，在函数结束时释放。

D 编译不过，编译器会提醒你漏了 `forget`

能编译。返回一个裸指针本身不需要 `unsafe`，编译器不追踪它指向的内存还在不在，这正是要人来担保的地方。

选择题

线性内存扩容之后，之前创建的 Uint8Array 视图会怎样？

- A 自动跟着变大，照常使用
- B 还能读，只是读到的是扩容前的旧数据
- C 原来的 ArrayBuffer 被分离，旧视图长度变成 0，读不到数据
- D 浏览器会提示内存不足

选择题

线性内存扩容之后，之前创建的 Uint8Array 视图会怎样？

A 自动跟着变大，照常使用

不会。扩容后原来的 ArrayBuffer 被分离了，旧视图不会跟着走。

B 还能读，只是读到的是扩容前的旧数据

读不到。旧视图的长度变成 0，读出来是空的。

C 原来的 ArrayBuffer 被分离，旧视图长度变成 0，读不到数据

对。所以 proto.js 不缓存视图，每次读写都基于当前的 memory.buffer 现取。

D 浏览器会提示内存不足

扩容就是为了要更多内存，和内存不足没关系。问题出在旧视图上。

想一想

手机通过 http 打开页面时，`crypto.subtle` 用不了，可 PAKE 需要密码学安全的随机数。WASM 里的随机数从哪来？为什么在 http 页面里也行得通？

想一想

手机通过 http 打开页面时，`crypto.subtle` 用不了，可 PAKE 需要密码学安全的随机数。WASM 里的随机数从哪来？为什么在 http 页面里也行得通？

WASM 声明一个导入函数 `ls_random`（来自 `env` 模块），注册给 `getrandom` 库，需要随机数时就调它；JS 加载 WASM 时用 `crypto.getRandomValues` 实现它。`crypto.getRandomValues` 是密码学安全的随机源，在 http 页面里也能用，受安全上下文限制的只有 `crypto.subtle`。

这一集我们学了

- ① JS 和 WASM 之间只能传数字，大块数据靠线性内存加地址
- ① unsafe 圈出编译器管不了的地方，每一处写 SAFETY 说明为什么成立
- ① forget 把所有权交给 JS，ls_free 再收回来，申请和释放一一对应
- ① Option + take 让 PAKE 状态只能用一次
- ① JS 端：每次现取视图，u64 拆成两个 u32 传

下一集

下一集：服务端，async/await、axum 与共享状态